Observabilidade em Aplicações .net

Definição

Observabilidade é a capacidade de compreender o estado interno de um sistema através das saídas que ele produz. Em aplicações, significa poder responder perguntas arbitrárias sobre o comportamento do sistema sem precisar modificar código ou fazer deploy de novas instrumentações.

Papel e Importância

Por que é importante?

1. **Visibilidade Total**: Permite entender o que está acontecendo internamente na aplicação
2. **Debugging Proativo**: Identifica problemas antes que afetem usuários
3. **Otimização**: Identifica gargalos de performance e oportunidades de melhoria
4. **Confiabilidade**: Aumenta a disponibilidade e resiliência do sistema
5. **Business Intelligence**: Correlaciona métricas técnicas com impacto no negócio

Como ajuda no desenvolvimento?

- **Feedback rápido** sobre mudanças de código
- **Identificação de bugs** em ambientes reais
- **Testes de performance** contínuos
- **Compreensão de dependências** entre serviços
- **Validação de hipóteses** sobre comportamento do sistema

Em War Rooms e Incidentes Críticos

Durante incidentes, observabilidade é **crucial**:

- ⏱️ **Reduz MTTR** (Mean Time To Recovery)
- 🔍 **Acelera diagnóstico** com correlação de dados
- 📊 **Visualiza impacto** em tempo real
- 🎯 **Identifica causa raiz** rapidamente
- 📈 **Prioriza ações** baseado em impacto no negócio

Em uma **aplicação de consórcios** onde vender é prioridade, você pode:

- Detectar quando vendas param de acontecer
- Identificar qual serviço/API está causando o problema
- Ver exatamente qual endpoint está falhando

- Correlacionar erros com transações de negócio

Monitoramento vs Observabilidade

Aspecto	Monitoramento	Observabilidade
Abordagem	Reativa - "O que sabemos que pode falhar?"	Proativa - "O que está acontecendo?"
Perguntas	Predefinidas (dashboards fixos)	Arbitrárias (exploração ad-hoc)
Foco	Sintomas conhecidos	Investigação de unknowns
Dados	Métricas agregadas	Traces, logs, métricas correlacionados
Alertas	Baseados em thresholds	Baseados em anomalias e contexto

Analogia: Monitoramento é como ter termômetros em pontos específicos de uma casa. Observabilidade é como ter câmeras térmicas que mostram todo o fluxo de calor.

Instrumentando Aplicação .NET para Observabilidade

Arquitetura da Solução

```
Aplicação .NET
├── OpenTelemetry SDK
│   ├── Traces → Grafana Tempo
│   ├── Metrics → Prometheus
│   └── Logs → Loki (via Alloy)
└── Health Checks → Métricas customizadas
```

Passo 1: Instalação de Pacotes

```
dotnet add package OpenTelemetry.Extensions.Hosting

dotnet add package OpenTelemetry.Instrumentation.AspNetCore

dotnet add package OpenTelemetry.Instrumentation.Http

dotnet add package OpenTelemetry.Instrumentation.SqlClient

dotnet add package OpenTelemetry.Exporter.Prometheus.AspNetCore
```

dotnet add package OpenTelemetry.Exporter.OpenTelemetryProtocol

dotnet add package Serilog.AspNetCore

dotnet add package Serilog.Sinks.Console

dotnet add package Serilog.Formatting.Compact

dotnet add package Serilog.Enrichers.Environment

dotnet add package Serilog.Enrichers.Thread

dotnet add package Microsoft.Extensions.Diagnostics.HealthChecks

dotnet add package AspNetCore.HealthChecks.SqlServer

dotnet add package AspNetCore.HealthChecks.Uris

Passo 2: Configuração do Program.cs

```
using System.Diagnostics;
using System.Diagnostics.Metrics;
using Microsoft.AspNetCore.Diagnostics.HealthChecks;
using OpenTelemetry.Logs;
using OpenTelemetry.Metrics;
using OpenTelemetry.Resources;
using OpenTelemetry.Trace;
using Serilog;
using Serilog.Formatting.Compact;

var builder = WebApplication.CreateBuilder(args);

// ========================================
// 1. CONFIGURAÇÃO DE LOGS ESTRUTURADOS
// ========================================

Log.Logger = new LoggerConfiguration()
    .ReadFrom.Configuration(builder.Configuration)
    .Enrich.FromLogContext()
    .Enrich.WithMachineName()
    .Enrich.WithThreadId()
    .Enrich.WithProperty("Application", "ConsorcioApp")
    .Enrich.WithProperty("Environment",
```

```
builder.Environment.EnvironmentName)
    .WriteTo.Console(new CompactJsonFormatter())
    .CreateLogger();

builder.Host.UseSerilog();

// ========================================
// 2. CONFIGURAÇÃO DE OBSERVABILIDADE
// ========================================

var serviceName = "consorcio-api";
var serviceVersion = "1.0.0";

// Resource compartilhado para todos os sinais
var resourceBuilder = ResourceBuilder
    .CreateDefault()
    .AddService(serviceName, serviceVersion: serviceVersion)
    .AddAttributes(new Dictionary<string, object>
    {
        ["deployment.environment"] =
builder.Environment.EnvironmentName,
        ["business.domain"] = "vendas-consorcio"
    });

// ========================================
// 3. MÉTRICAS CUSTOMIZADAS
// ========================================

var meterProvider =
builder.Services.AddSingleton<ConsorcioMetrics>();

// ========================================
// 4. OPENTELEMETRY - TRACES
// ========================================

builder.Services.AddOpenTelemetry()
    .ConfigureResource(resource => resource.AddService(serviceName))
    .WithTracing(tracing =>
    {
        tracing
            .SetResourceBuilder(resourceBuilder)
            .AddAspNetCoreInstrumentation(options =>
```

```
            {
                options.RecordException = true;
                options.EnrichWithHttpRequest = (activity,
httpRequest) =>
                {
                    activity.SetTag("http.request.user_agent",
httpRequest.Headers.UserAgent.ToString());
                };
                options.EnrichWithHttpResponse = (activity,
httpResponse) =>
                {
                    activity.SetTag("http.response.status_code",
httpResponse.StatusCode);
                };
            })
            .AddHttpClientInstrumentation(options =>
            {
                options.RecordException = true;
                options.EnrichWithHttpRequestMessage = (activity,
httpRequest) =>
                {
                    activity.SetTag("http.request.method",
httpRequest.Method.ToString());
                };
            })
            .AddSqlClientInstrumentation(options =>
            {
                options.SetDbStatementForText = true;
                options.RecordException = true;
                options.EnableConnectionLevelAttributes = true;
            })
            .AddSource("ConsorcioApp.*")
            .AddOtlpExporter(options =>
            {
                options.Endpoint = new
Uri(builder.Configuration["OpenTelemetry:Tempo:Endpoint"]
                    ?? "http://localhost:4317");
            });
    })
    // ========================================
    // 5. OPENTELEMETRY - METRICS
    // ========================================
```

```
    .WithMetrics(metrics =>
    {
        metrics
            .SetResourceBuilder(resourceBuilder)
            .AddAspNetCoreInstrumentation()
            .AddHttpClientInstrumentation()
            .AddRuntimeInstrumentation()
            .AddProcessInstrumentation()
            .AddMeter("ConsorcioApp.*")
            .AddPrometheusExporter();
    });

// ========================================
// 6. HEALTH CHECKS
// ========================================

builder.Services.AddHealthChecks()
    .AddSqlServer(
        connectionString:
builder.Configuration.GetConnectionString("DefaultConnection")!,
        name: "database",
        tags: new[] { "db", "sql", "critical" })
    .AddUrlGroup(
        new
Uri(builder.Configuration["ExternalApis:CreditAnalysis:Url"]!),
        name: "credit-analysis-api",
        tags: new[] { "api", "external", "critical" })
    .AddUrlGroup(
        new
Uri(builder.Configuration["ExternalApis:PaymentGateway:Url"]!),
        name: "payment-gateway-api",
        tags: new[] { "api", "external", "critical" })
    .AddCheck<CustomHealthCheck>("custom-business-check", tags:
new[] { "business" });

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// ========================================
```

```
// 7. MIDDLEWARE DE MÉTRICAS CUSTOMIZADAS
// ========================================

app.UseMiddleware<BusinessMetricsMiddleware>();

// ========================================
// 8. ENDPOINTS DE OBSERVABILIDADE
// ========================================

// Prometheus metrics endpoint
app.MapPrometheusScrapingEndpoint("/metrics");

// Health checks endpoints
app.MapHealthChecks("/health", new HealthCheckOptions
{
    ResponseWriter = async (context, report) =>
    {
        context.Response.ContentType = "application/json";
        var result = System.Text.Json.JsonSerializer.Serialize(new
        {
            status = report.Status.ToString(),
            timestamp = DateTime.UtcNow,
            duration = report.TotalDuration,
            checks = report.Entries.Select(e => new
            {
                name = e.Key,
                status = e.Value.Status.ToString(),
                description = e.Value.Description,
                duration = e.Value.Duration,
                exception = e.Value.Exception?.Message,
                data = e.Value.Data
            })
        });
        await context.Response.WriteAsync(result);
    }
});

// Liveness probe (simples)
app.MapHealthChecks("/health/live", new HealthCheckOptions
{
    Predicate = _ => false
});
```

```
// Readiness probe (verifica dependências críticas)
app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
    Predicate = check => check.Tags.Contains("critical")
});

// Logs endpoint (para debug - REMOVER EM PRODUÇÃO ou proteger)
app.MapGet("/logs", () => Results.Ok(new
{
    message = "Logs são enviados via Serilog para stdout/Loki",
    configuration = "Verifique Loki para visualizar logs
estruturados"
}));

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

try
{
    Log.Information("Iniciando aplicação ConsorcioApp");
    app.Run();
}
catch (Exception ex)
{
    Log.Fatal(ex, "Aplicação falhou ao iniciar");
    throw;
}
finally
{
    Log.CloseAndFlush();
}
```

Passo 3: Classe de Métricas Customizadas

```
using System.Diagnostics.Metrics;

public class ConsorcioMetrics
{
    private readonly Meter _meter;

    // Contadores
    public Counter<long> VendasRealizadas { get; }
    public Counter<long> VendasFalhadas { get; }
    public Counter<long> PropostasCriadas { get; }
    public Counter<long> AnaliseCreditoSolicitada { get; }

    // Histogramas
    public Histogram<double> TempoProcessamentoVenda { get; }
    public Histogram<double> ValorVenda { get; }
    public Histogram<double> TempoConsultaCredito { get; }

    // Gauges (via ObservableGauge)
    public ObservableGauge<int> PropostasEmAnalise { get; }
    public ObservableGauge<int> VendasDia { get; }

    private int _propostasEmAnaliseCount = 0;
    private int _vendasDiaCount = 0;

    public ConsorcioMetrics(IMeterFactory meterFactory)
```

```
    {
        _meter = meterFactory.Create("ConsorcioApp.Business");

        // Contadores
        VendasRealizadas = _meter.CreateCounter<long>(
            "consorcio.vendas.realizadas",
            unit: "vendas",
            description: "Total de vendas realizadas com sucesso");

        VendasFalhadas = _meter.CreateCounter<long>(
            "consorcio.vendas.falhadas",
            unit: "vendas",
            description: "Total de vendas que falharam");

        PropostasCriadas = _meter.CreateCounter<long>(
            "consorcio.propostas.criadas",
            unit: "propostas",
            description: "Total de propostas criadas");

        AnaliseCreditoSolicitada = _meter.CreateCounter<long>(
            "consorcio.credito.analises",
            unit: "analises",
            description: "Total de análises de crédito
solicitadas");

        // Histogramas
        TempoProcessamentoVenda = _meter.CreateHistogram<double>(
            "consorcio.vendas.duracao",
            unit: "ms",
```

```
        description: "Tempo de processamento de uma venda");

    ValorVenda = _meter.CreateHistogram<double>(
        "consorcio.vendas.valor",
        unit: "BRL",
        description: "Valor monetário das vendas");

    TempoConsultaCredito = _meter.CreateHistogram<double>(
        "consorcio.credito.duracao",
        unit: "ms",
        description: "Tempo de consulta à API de crédito");

    // Gauges observáveis
    PropostasEmAnalise = _meter.CreateObservableGauge(
        "consorcio.propostas.em_analise",
        () => _propostasEmAnaliseCount,
        unit: "propostas",
        description: "Número atual de propostas em análise");

    VendasDia = _meter.CreateObservableGauge(
        "consorcio.vendas.dia_atual",
        () => _vendasDiaCount,
        unit: "vendas",
        description: "Total de vendas realizadas hoje");
}

public void IncrementarPropostasEmAnalise() =>
    Interlocked.Increment(ref _propostasEmAnaliseCount);
```

```
    public void DecrementarPropostasEmAnalise() =>
        Interlocked.Decrement(ref _propostasEmAnaliseCount);

    public void IncrementarVendasDia() =>
        Interlocked.Increment(ref _vendasDiaCount);

    public void ResetarVendasDia() =>
        Interlocked.Exchange(ref _vendasDiaCount, 0);
}
```

Passo 4: Middleware para Métricas de Negócio

```
using System.Diagnostics;

public class BusinessMetricsMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ConsorcioMetrics _metrics;
    private readonly ILogger<BusinessMetricsMiddleware> _logger;

    public BusinessMetricsMiddleware(
        RequestDelegate next,
        ConsorcioMetrics metrics,
        ILogger<BusinessMetricsMiddleware> logger)
    {
        _next = next;
        _metrics = metrics;
        _logger = logger;
```

```
}

public async Task InvokeAsync(HttpContext context)
{
    var sw = Stopwatch.StartNew();
    var path = context.Request.Path.Value?.ToLowerInvariant() ?? "";

    try
    {
        await _next(context);

        sw.Stop();

        // Registrar métricas baseadas no endpoint
        var tags = new TagList
        {
            { "endpoint", path },
            { "method", context.Request.Method },
            { "status_code", context.Response.StatusCode }
        };

        // Métricas específicas de negócio
        if (path.Contains("/vendas") && context.Response.StatusCode == 200)
        {
            _metrics.VendasRealizadas.Add(1, tags);
            _metrics.IncrementarVendasDia();

            _logger.LogInformation(
```

```
                "Venda realizada com sucesso em {Duration}ms - Endpoint: {Endpoint}",
                sw.ElapsedMilliseconds,
                path);
            }
            else if (path.Contains("/vendas") && context.Response.StatusCode >= 400)
            {
                _metrics.VendasFalhadas.Add(1, tags);

                _logger.LogWarning(
                    "Venda falhou com status {StatusCode} em {Duration}ms - Endpoint: {Endpoint}",
                    context.Response.StatusCode,
                    sw.ElapsedMilliseconds,
                    path);
            }

            if (path.Contains("/propostas") && context.Response.StatusCode == 201)
            {
                _metrics.PropostasCriadas.Add(1, tags);
            }
        }
        catch (Exception ex)
        {
            sw.Stop();

            _logger.LogError(
                ex,
                "Erro não tratado no endpoint {Endpoint} após {Duration}ms",
```

```
            path,
            sw.ElapsedMilliseconds);

        throw;
    }
  }
}
```

Passo 5: Health Check Customizado

```
using Microsoft.Extensions.Diagnostics.HealthChecks;

public class CustomHealthCheck : IHealthCheck
{
    private readonly ILogger<CustomHealthCheck> _logger;
    private readonly ConsorcioMetrics _metrics;

    public CustomHealthCheck(
        ILogger<CustomHealthCheck> logger,
        ConsorcioMetrics metrics)
    {
        _logger = logger;
        _metrics = metrics;
    }

    public Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context,
        CancellationToken cancellationToken = default)
    {
        try
```

```
{
    // Exemplo: verificar se há propostas travadas há muito tempo
    // Em produção, isso seria uma query real
    var propostas EmAnalise = 0; // Simular consulta

    if (propostas EmAnalise > 1000)
    {
        return Task.FromResult(
            HealthCheckResult.Degraded(
                "Muitas propostas em análise",
                data: new Dictionary<string, object>
                {
                    { "propostas_em_analise", propostas EmAnalise }
                }));
    }

    return Task.FromResult(
        HealthCheckResult.Healthy(
            "Checks de negócio OK",
            data: new Dictionary<string, object>
            {
                { "propostas_em_analise", propostas EmAnalise }
            }));
}
catch (Exception ex)
{
    _logger.LogError(ex, "Erro ao executar health check customizado");
    return Task.FromResult(
```

```
            HealthCheckResult.Unhealthy(
                "Erro ao verificar saúde do sistema",
                ex));
        }
    }
}
```

Passo 6: Exemplo de Controller Instrumentado

```
using System.Diagnostics;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class VendasController : ControllerBase
{
    private static readonly ActivitySource ActivitySource =
new("ConsorcioApp.Vendas");

    private readonly ILogger<VendasController> _logger;
    private readonly ConsorcioMetrics _metrics;
    private readonly IHttpClientFactory _httpClientFactory;

    public VendasController(
        ILogger<VendasController> logger,
        ConsorcioMetrics metrics,
        IHttpClientFactory httpClientFactory)
    {
        _logger = logger;
```

```
        _metrics = metrics;
        _httpClientFactory = httpClientFactory;
    }

    [HttpPost]
    public async Task<IActionResult> RealizarVenda([FromBody] VendaRequest
request)
    {
        // Criar span customizado
        using var activity = ActivitySource.StartActivity("ProcessarVenda");
        activity?.SetTag("cliente.cpf", request.CpfCliente);
        activity?.SetTag("produto.tipo", request.TipoProduto);
        activity?.SetTag("valor", request.Valor);

        var sw = Stopwatch.StartNew();

        try
        {
            _logger.LogInformation(
                "Iniciando processamento de venda - Cliente: {CpfCliente}, Valor: {Valor}",
                request.CpfCliente,
                request.Valor);

            // 1. Criar proposta
            using (var propostaActivity = ActivitySource.StartActivity("CriarProposta"))
            {
                await CriarPropostaAsync(request);
                _metrics.PropostasCriadas.Add(1, new TagList
```

```
    {
        { "tipo_produto", request.TipoProduto }
    });
}

// 2. Consultar análise de crédito
using (var creditoActivity = ActivitySource.StartActivity("ConsultarCredito"))
{
    var swCredito = Stopwatch.StartNew();
    var aprovado = await ConsultarAnaliseCredito(request.CpfCliente);
    swCredito.Stop();

    _metrics.AnaliseCreditoSolicitada.Add(1);
    _metrics.TempoConsultaCredito.Record(
        swCredito.ElapsedMilliseconds,
        new TagList { { "aprovado", aprovado } });

    creditoActivity?.SetTag("credito.aprovado", aprovado);

    if (!aprovado)
    {
        _logger.LogWarning(
            "Crédito não aprovado para cliente {CpfCliente}",
            request.CpfCliente);

        activity?.SetStatus(ActivityStatusCode.Error, "Crédito não aprovado");

        return BadRequest(new { message = "Crédito não aprovado" });
```

```
        }
    }

    // 3. Finalizar venda
    using (var finalizacaoActivity = ActivitySource.StartActivity("FinalizarVenda"))
    {
        await FinalizarVendaAsync(request);
    }

    sw.Stop();

    // Registrar métricas de sucesso
    _metrics.VendasRealizadas.Add(1, new TagList
    {
        { "tipo_produto", request.TipoProduto },
        { "canal", request.Canal }
    });

    _metrics.TempoProcessamentoVenda.Record(sw.ElapsedMilliseconds);
    _metrics.ValorVenda.Record(request.Valor, new TagList
    {
        { "tipo_produto", request.TipoProduto }
    });

    _logger.LogInformation(
        "Venda realizada com sucesso em {Duration}ms - Cliente: {CpfCliente},
Valor: {Valor}",
        sw.ElapsedMilliseconds,
```

```
        request.CpfCliente,
        request.Valor);

    activity?.SetStatus(ActivityStatusCode.Ok);

    return Ok(new { message = "Venda realizada com sucesso" });
}
catch (Exception ex)
{
    sw.Stop();

    _metrics.VendasFalhadas.Add(1, new TagList
    {
        { "tipo_produto", request.TipoProduto },
        { "erro_tipo", ex.GetType().Name }
    });

    _logger.LogError(
        ex,
        "Erro ao processar venda após {Duration}ms - Cliente: {CpfCliente}",
        sw.ElapsedMilliseconds,
        request.CpfCliente);

    activity?.SetStatus(ActivityStatusCode.Error, ex.Message);
    activity?.RecordException(ex);

    return StatusCode(500, new { message = "Erro ao processar venda" });
}
```

```
}

[HttpGet("mais-vendidos")]
public async Task<IActionResult> ObterProdutosMaisVendidos()
{
    using var activity = ActivitySource.StartActivity("ObterProdutosMaisVendidos");

    try
    {
        // Simular consulta ao banco
        var produtos = await ObterProdutosAsync();

        return Ok(produtos);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Erro ao obter produtos mais vendidos");
        activity?.RecordException(ex);
        return StatusCode(500);
    }
}

private async Task CriarPropostaAsync(VendaRequest request)
{
    // Simular criação de proposta
    await Task.Delay(50);
    _metrics.IncrementarPropostasEmAnalise();
}
```

```
private async Task<bool> ConsultarAnaliseCredito(string cpf)
{
    using var activity = ActivitySource.StartActivity("HttpCall.CreditAnalysisApi");

    try
    {
        var client = _httpClientFactory.CreateClient();
        var response = await client.GetAsync($"http://credit-api/analyze/{cpf}");

        activity?.SetTag("http.status_code", (int)response.StatusCode);

        return response.IsSuccessStatusCode;
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Erro ao consultar API de crédito");
        activity?.RecordException(ex);
        throw;
    }
}

private async Task FinalizarVendaAsync(VendaRequest request)
{
    // Simular finalização
    await Task.Delay(100);
    _metrics.DecrementarPropostasEmAnalise();
}
```

```
    private async Task<List<object>> ObterProdutosAsync()
    {
        // Simular query lenta
        await Task.Delay(200);
        return new List<object>();
    }
}

public record VendaRequest(
    string CpfCliente,
    string TipoProduto,
    decimal Valor,
    string Canal);
```

Passo 7: Configuração appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "System.Net.Http.HttpClient": "Warning"
    }
  },
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=ConsorcioDb;User
Id=sa;Password=YourPassword;TrustServerCertificate=True"
```

```
},
"ExternalApis": {
  "CreditAnalysis": {
    "Url": "http://credit-api:8080/health"
  },
  "PaymentGateway": {
    "Url": "http://payment-api:8080/health"
  }
},
"OpenTelemetry": {
  "Tempo": {
    "Endpoint": "http://localhost:4317"
  }
},
"Serilog": {
  "Using": [ "Serilog.Sinks.Console" ],
  "MinimumLevel": {
    "Default": "Information",
    "Override": {
      "Microsoft": "Warning",
      "System": "Warning"
    }
  },
  "WriteTo": [
    {
      "Name": "Console",
      "Args": {
```

```
        "formatter": "Serilog.Formatting.Compact.CompactJsonFormatter, Serilog.Formatting.Compact"
      }
    }
  ],
  "Enrich": [ "FromLogContext", "WithMachineName", "WithThreadId" ]
 }
}
```

Passo 8: Docker Compose para Stack de Observabilidade

```
version: '3.8'

services:
  # Aplicação .NET
  consorcio-api:
    build: .
    ports:
      - "5000:8080"
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - OpenTelemetry__Tempo__Endpoint=http://tempo:4317
      - ConnectionStrings__DefaultConnection=Server=sqlserver;Database=ConsorcioDb;User Id=sa;Password=YourStrong@Password;TrustServerCertificate=True
```